Phew! That was some YAML wrangling again, so let's get that applied. In a new terminal session, go to `ch05/clickhouse/`, and then execute (self-contained, no need for an external load generator) the following:

```
docker-compose -f docker-compose.yaml up
```

Now you can go to http://localhost:8123/play, which is ClickHouse's playground, and enter the following SQL query (for reference, on my machine, it takes some milliseconds to read around 1,000 rows):

```
SELECT
  Timestamp as ts,
  Body AS payload,
  JSONExtractString(Body, 'traceID') AS traceID
FROM otel_logs
WHERE
  match(payload, 'invoked') AND
  ts >= NOW() - INTERVAL 30 SECOND
```

We pull out certain columns of interest, especially the associated trace ID.

We do a regex match against the body of the log message, requiring it to contain the string invoked.

We limit the output to the last 30 seconds.

The table the OpenTelemetry ClickHouse exporter set up for us to query.

The result of the SQL query execution is what you see in figure 5.6. Note the query stats in the right area above the output table.

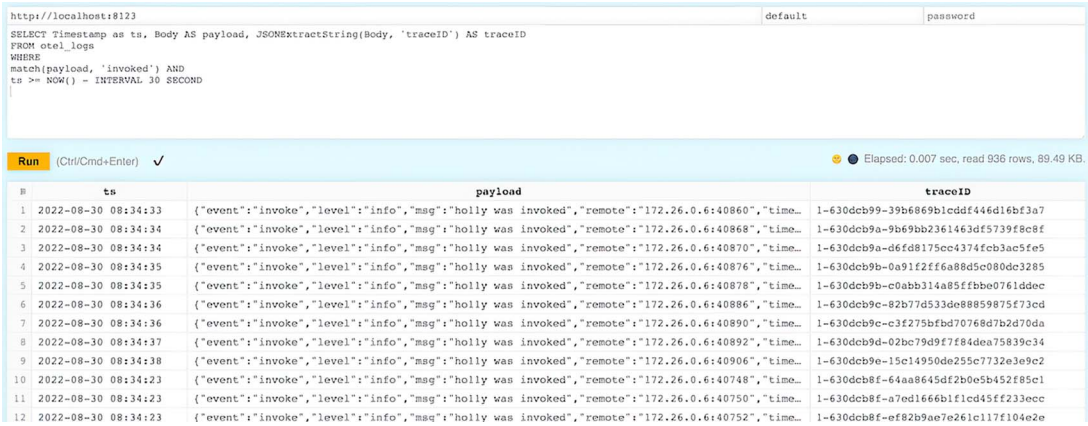


Figure 5.6 The ClickHouse Web UI playground, executing our SQL query, and showing the results

TIP If you find yourself working more with ClickHouse, then I'd recommend you check out ClickCat (https://github.com/clickcat-project/ClickCat), a nice and simple user interface that lets you search, explore, and visualize data in ClickHouse.